

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Dessy Nurulita

NIM. 2310817220024

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Dessy Nurulita
NIM : 2310817220024

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	6
B. Output Program	20
C. Pembahasan	25
D. Tautan Git	42
SOAL 2.....	43

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 (List Portrait).....	20
Gambar 2. Screenshot Hasil Jawaban Soal 1 (Detail Portrait)	21
Gambar 3. Screenshot Hasil Jawaban Soal 1 (List Landscape)	22
Gambar 4. Screenshot Hasil Jawaban Soal 1 (Detail Landscape)	22
Gambar 5. Screenshot Hasil Jawaban Soal 1 (Step Into)	23
Gambar 6. Screenshot Hasil Jawaban Soal 1 (Step Out).....	23
Gambar 7. Screenshot Hasil Jawaban Soal 1 (Step Over).....	24
Gambar 8. Screenshot Hasil Jawaban Soal 1 (Logcat).....	24
Gambar 9. Screenshot Hasil Jawaban Soal 1 (Validasi Logcat)	25

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1 (MainActivity.kt)	6
Tabel 2. Source Code Jawaban Soal 1 (Constellation.kt)	7
Tabel 3. Source Code Jawaban Soal 1 (ConstellationData.kt)	7
Tabel 4. Source Code Jawaban Soal 1 (HomeFragment.kt)	9
Tabel 5. Source Code Jawaban Soal 1 (DetailFragment.kt)	10
Tabel 6. Source Code Jawaban Soal 1 (ListConstellationsAdapter.kt)	12
Tabel 7. Source Code Jawaban Soal 1 (activity_main.xml)	13
Tabel 8. Source Code Jawaban Soal 1 (fragment_home.xml)	14
Tabel 9. Source Code Jawaban Soal 1 (fragment_detail.xml)	14
Tabel 10. Source Code Jawaban Soal 1 (star_item.xml)	15
Tabel 11. Source Code Jawaban Soal 1 (nav_graph.xml)	17
Tabel 12. Source Code Jawaban Soal 1 (styles.xml)	18
Tabel 13. Source Code Jawaban Soal 1 (MainViewModel.kt)	18
Tabel 14. Source Code Jawaban Soal 1 (MainViewModelFactory.kt)	19

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

1. MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 (MainActivity.kt)

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.navigation.fragment.NavHostFragment
6	import androidx.navigation.ui.setupActionBarWithNavController
7	import com.example. modul4.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	
11	private lateinit var binding: ActivityMainBinding
12	
13	override fun onCreate(savedInstanceState: Bundle?) {
14	super.onCreate(savedInstanceState)
15	
16	binding = ActivityMainBinding.inflate(layoutInflater)
17	setContentView(binding.root)

18	
19	setSupportActionBar(binding.toolbar)
20	
21	val navHostFragment = supportFragmentManager
22	.findFragmentById(R.id.nav_host_fragment) as
23	NavHostFragment
24	val navController = navHostFragment.navController
25	
26	setUpActionBarWithNavController(navController)
27	}
28	
29	override fun onSupportNavigateUp(): Boolean {
30	val navHostFragment = supportFragmentManager
31	.findFragmentById(R.id.nav_host_fragment) as
32	NavHostFragment
33	return navHostFragment.navController.navigateUp()
34	super.onSupportNavigateUp()
	}
	}

2. Constellation.kt

Tabel 2. Source Code Jawaban Soal 1 (Constellation.kt)

1	package com.example.modul4
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Constellation(
8	val name: String,
9	val description: String,
10	val year: String,
11	val imageResId: Int,
12	val webUrl: String
13	) : Parcelable

3. ConstellationData.kt

Tabel 3. Source Code Jawaban Soal 1 (ConstellationData.kt)

1	package com.example. modul4
2	
3	object ConstellationData {
4	val listConstellations = listOf(
5	Constellation(
6	name = "Hydra",
7	description = "Hydra, alias sang ular air (The
8	Water Serpent), adalah rasi bintang terpanjang dan terbesar

9	di langit malam. Luasnya sebesar 1.303 derajat persegi
10	(~3,16% dari seluruh langit malam). Bentuknya menyerupai ular
11	air dan membentang dari selatan Cancer hingga Libra. Meski
12	ukurannya besar, hanya sedikit bintang terang yang terlihat.
13	Hydra dikenal dalam mitologi Yunani sebagai ular berkepala
14	banyak yang dikalahkan oleh Herkules.",
15	imageResId = R.drawable.hydra,
16	year = "420 juta tahun",
17	webUrl =
18	"https://en.m.wikipedia.org/wiki/Hydra_(constellation)"
19	),
20	Constellation(
21	name = "Virgo",
22	description = "Virgo, alias sang perawan suci
23	(The Maiden), adalah rasi bintang terbesar kedua dan salah
24	satu dari zodiak. Luasnya sebesar 1.294 derajat persegi
25	(~3,14% dari langit malam). Bintang terangnya, Spica, sangat
26	mudah dikenali. Virgo melambangkan dewi panen dan kesuburan
27	dalam berbagai mitologi, termasuk Yunani dan Romawi. Rasi ini
28	juga penting dalam astronomi karena menjadi lokasi
29	superklaster galaksi yang dinamakan Virgo Cluster.",
30	imageResId = R.drawable.virgo,
31	year = "12-400 juta tahun",
32	webUrl =
33	"https://en.m.wikipedia.org/wiki/Virgo_(constellation)"
34	),
35	Constellation(
36	name = "Ursa Major",
37	description = "Ursa Major, alias sang beruang
38	besar (The Great Bear), adalah salah satu rasi bintang paling
39	dikenal karena mengandung asterisma Big Dipper (Gayung
40	Besar). Luasnya sebesar 1.280 derajat persegi (~3,11% dari
41	langit malam). Rasi ini penting dalam navigasi karena dua
	bintangnya menunjuk ke Polaris (Bintang Utara).",
	imageResId = R.drawable.ursa_major,
	year = "300 juta - 1 miliar tahun",
	webUrl =
	"https://en.m.wikipedia.org/wiki/Ursa_Major"
	),
	Constellation(
	name = "Cetus",
	description = "Cetus, alias sang monster laut
	(The Sea Monster), dikenal sebagai 'Ikan Paus' atau 'Monster
	Laut' dalam mitologi Yunani. Luasnya sebesar 1.231 derajat
	persegi (~2,99% dari langit malam). Rasi ini cukup besar dan
	terletak di dekat rasi Pisces dan Eridanus. Meskipun luas,
	Cetus tidak memiliki banyak bintang terang, namun menjadi
	rumah bagi salah satu bintang variabel terkenal, Mira.",
	imageResId = R.drawable.cetus,
	year = "2 miliar tahun",
	webUrl = "https://en.m.wikipedia.org/wiki/Cetus"
	),

	<pre> Constellation(name = "Hercules", description = "Hercules, alias sang pahlawan kuat, dinamai dari pahlawan mitologi Yunani, yaitu 'Heracles'. Luasnya sebesar 1.225 derajat persegi (~2,97% dari langit malam). Bentuknya tidak mudah dikenali, tapi rasi ini memiliki formasi yang dikenal sebagai \"Keystone\" (batu penjuru). Salah satu daya tariknya adalah Gugus Bola M13, salah satu gugus bintang paling terang yang bisa dilihat dengan teleskop kecil.", imageResId = R.drawable.hercules, year = " 11 miliar tahun", webUrl = "https://en.m.wikipedia.org/wiki/Hercules_(constellation)"),) </pre>
--	---

4. HomeFragment.kt

Tabel 4. Source Code Jawaban Soal 1 (HomeFragment.kt)

1	package com.example.modul4
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.LifecycleScope
12	import androidx.navigation.fragment.findNavController
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import com.example.modul4.databinding.FragmentHomeBinding
15	import kotlinx.coroutines.flow.collectLatest
16	import kotlinx.coroutines.launch
17	
18	class HomeFragment : Fragment() {
19	
20	private var _binding: FragmentHomeBinding? = null
21	private val binding get() = _binding!!
22	
23	private val viewModel: MainViewModel by viewModels {
24	MainViewModelFactory() }
25	
26	private lateinit var adapter: ListConstellationsAdapter
27	
28	override fun onCreateView(
29	inflater: LayoutInflater, container: ViewGroup?,

30	savedInstanceState: Bundle?
31	): View {
32	_binding = FragmentHomeBinding.inflate(inflater,
33	container, false)
34	return binding.root
35	}
36	
37	override fun onCreateView(view: View,
38	savedInstanceState: Bundle?) {
39	super.onCreateView(view, savedInstanceState)
40	
41	adapter = ListConstellationsAdapter(
42	onWebClick = { constellation ->
43	viewModel.onWebClicked(constellation)
44	val intent = Intent(Intent.ACTION_VIEW,
45	Uri.parse(constellation.webUrl))
46	startActivity(intent)
47	},
48	onDetailClick = { constellation ->
49	viewModel.onDetailClicked(constellation)
50	val action =
51	HomeFragmentDirections.actionHomeFragmentToDetailFragment(co
42	nstellations)
53	findNavController().navigate(action)
54	}
55	)
56	
57	binding.rvStars.layoutManager =
58	LinearLayoutManager(requireContext())
59	binding.rvStars.adapter = adapter
60	
61	lifecycleScope.launch {
62	viewModel.constellations.collectLatest {
63	adapter.submitList(it)
64	}
65	}
	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

5. DetailFragment.kt

Tabel 5. Source Code Jawaban Soal 1 (DetailFragment.kt)

1	package com.example.modul4
2	
3	import android.os.Bundle

```

4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.navArgs
9 import com.bumptech.glide.Glide
10 import com.example.modul4.databinding.FragmentDetailBinding
11
12 class DetailFragment : Fragment() {
13
14     private var _binding: FragmentDetailBinding? = null
15     private val binding get() = _binding!!
16     private val args: DetailFragmentArgs by navArgs()
17
18     override fun onCreateView(
19         inflater: LayoutInflater, container: ViewGroup?,
20         savedInstanceState: Bundle?
21     ): View {
22         _binding = FragmentDetailBinding.inflate(inflater,
23 container, false)
24         return binding.root
25     }
26
27     override fun onViewCreated(view: View,
28 savedInstanceState: Bundle?) {
29         super.onViewCreated(view, savedInstanceState)
30
31         val data = args.constellation
32
33         binding.apply {
34             detailName.text = data.name
35             detailDescription.text = data.description
36             detailYear.text = data.year
37
38             Glide.with(this@DetailFragment).load(data.imageResId).into(d
39 etailImage)
40         }
41     }
42
43     override fun onDestroyView() {
44         super.onDestroyView()
45         _binding = null
46     }
47 }

```

6. ListConstellationsAdapter.kt

Tabel 6. Source Code Jawaban Soal 1 (ListConstellationsAdapter.kt)

```
1 package com.example.modul4
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.DiffUtil
6 import androidx.recyclerview.widget.ListAdapter
7 import androidx.recyclerview.widget.RecyclerView
8 import com.bumptech.glide.Glide
9 import com.example.modul4.databinding.StarItemBinding
10
11 class ListConstellationsAdapter(
12     private val onWebClick: (Constellation) -> Unit,
13     private val onDetailClick: (Constellation) -> Unit
14 ) : ListAdapter<Constellation,
15 ListConstellationsAdapter.ViewHolder>(DIFF_CALLBACK) {
16
17     inner class ViewHolder(val binding: StarItemBinding) :
18 RecyclerView.ViewHolder(binding.root)
19
20     override fun onCreateViewHolder(parent: ViewGroup,
21 viewType: Int): ViewHolder {
22         val binding =
23 StarItemBinding.inflate(LayoutInflater.from(parent.context),
24 parent, false)
25         return ViewHolder(binding)
26     }
27
28     override fun onBindViewHolder(holder: ViewHolder,
29 position: Int) {
30         val item = getItem(position)
31         with(holder.binding) {
32             tvName.text = item.name
33             tvDescription.text = item.description
34             tvYear.text = item.year
35
36 Glide.with(img.context).load(item.imageResId).into(img)
37
38             btnWeb.setOnClickListener { onWebClick(item) }
39             btnDetail.setOnClickListener {
40 onDetailClick(item) }
41         }
42     }
43
44     companion object {
45         val DIFF_CALLBACK = object :
46 DiffUtil.ItemCallback<Constellation>() {
47             override fun areItemsTheSame(oldItem:
48 Constellation, newItem: Constellation): Boolean {
```

	<pre> return oldItem.name == newItem.name } override fun areContentsTheSame(oldItem: Constellation, newItem: Constellation): Boolean { return oldItem == newItem } } } </pre>
--	--

7. activity_main.xml

Tabel 7. Source Code Jawaban Soal 1 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	
9	<androidx.appcompat.widget.Toolbar
10	android:id="@+id/toolbar"
11	android:layout_width="match_parent"
12	android:layout_height="?attr/actionBarSize"
13	android:background="?attr/colorPrimary"
14	android:elevation="4dp"
15	app:titleTextColor="@android:color/white" />
16	
17	<androidx.fragment.app.FragmentContainerView
18	android:id="@+id/nav_host_fragment"
19	
20	android:name="androidx.navigation.fragment.NavHostFragment"
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	android:layout_marginTop="?attr/actionBarSize"
24	app:defaultNavHost="true"
25	app:navGraph="@navigation/nav_graph" />
	</androidx.coordinatorlayout.widget.CoordinatorLayout>

8. fragment_home.xml

Tabel 8. Source Code Jawaban Soal 1 (fragment_home.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:id="@+id/homeFragment"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="8dp">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/rvStars"
11	android:layout_width="match_parent"
12	android:layout_height="match_parent"
13	android:clipToPadding="false" />
14	</FrameLayout>

9. fragment_detail.xml

Tabel 9. Source Code Jawaban Soal 1 (fragment_detail.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/detailFragment"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:padding="16dp">
9	
10	<LinearLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:orientation="vertical"
14	android:gravity="center_horizontal">
15	
16	<ImageView
17	android:id="@+id/detailImage"
18	android:layout_width="350dp"
19	android:layout_height="400dp"
20	android:scaleType="centerCrop"
21	android:layout_marginBottom="16dp"
22	
23	app:shapeAppearanceOverlay="@style/RoundedImageView" />
24	
25	<TextView
26	android:id="@+id/detailName"
27	android:layout_width="wrap_content"
28	android:layout_height="wrap_content"

29	android:text="Hydra"
30	android:textSize="22sp"
31	android:textStyle="bold"
32	android:layout_marginBottom="8dp" />
33	
34	<TextView
35	android:id="@+id/detailYear"
36	android:layout_width="wrap_content"
37	android:layout_height="wrap_content"
38	android:text=""
39	android:textSize="16sp"
40	android:layout_marginBottom="16dp" />
41	
42	<TextView
43	android:id="@+id/detailDescription"
44	android:layout_width="match_parent"
45	android:layout_height="wrap_content"
46	android:text="This constellation is known
47	for..."
	android:textSize="16sp" />
	</LinearLayout>
	</ScrollView>

10. star_item.xml

Tabel 10. Source Code Jawaban Soal 1 (star_item.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:orientation="vertical"
8	android:padding="8dp">
9	
10	<androidx.cardview.widget.CardView
11	xmlns:card_view="http://schemas.android.com/apk/res-auto"
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:layout_margin="8dp"
15	card_view:cardCornerRadius="16dp"
16	card_view:cardElevation="4dp">
17	
18	<LinearLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:orientation="horizontal"
22	android:padding="8dp">
23	
24	

```

25 <com.google.android.material.imageview.ShapeableImageView
26     android:id="@+id/img"
27     android:layout_width="100dp"
28     android:layout_height="150dp"
29     android:scaleType="centerCrop"
30     android:layout_marginEnd="8dp"
31
32 app:shapeAppearanceOverlay="@style/RoundedImageView" />
33
34     <LinearLayout
35         android:layout_width="0dp"
36         android:layout_height="match_parent"
37         android:layout_weight="1"
38         android:orientation="vertical">
39
40         <LinearLayout
41             android:layout_width="match_parent"
42             android:layout_height="wrap_content"
43             android:orientation="horizontal"
44             android:gravity="center_vertical">
45
46             <TextView
47                 android:id="@+id/tvName"
48                 android:layout_width="0dp"
49                 android:layout_height="wrap_content"
50                 android:layout_weight="1"
51                 android:text="Orion"
52                 android:textStyle="bold"
53                 android:textSize="16sp" />
54
55             <TextView
56                 android:id="@+id/tvYear"
57                 android:layout_width="wrap_content"
58                 android:layout_height="wrap_content"
59                 android:text=""
60                 android:textSize="14sp" />
61         </LinearLayout>
62
63         <TextView
64             android:id="@+id/tvDescription"
65             android:layout_width="match_parent"
66             android:layout_height="wrap_content"
67             android:text="One of the brightest
68 constellations..."
69             android:textSize="14sp"
70             android:layout_marginTop="4dp"
71             android:maxLines="3"
72             android:ellipsize="end" />
73
74         <LinearLayout
75             android:layout_width="match_parent"
76             android:layout_height="wrap_content"

```


77	android:orientation="horizontal"
78	android:layout_marginTop="8dp">
79	
80	<Button
81	android:id="@+id/btnWeb"
82	android:layout_width="0dp"
83	android:layout_height="wrap_content"
84	android:layout_weight="1"
85	android:text="Website" />
86	
87	<Button
88	android:id="@+id/btnDetail"
89	android:layout_width="0dp"
90	android:layout_height="wrap_content"
91	android:layout_weight="1"
92	android:text="Detail"
93	android:layout_marginStart="8dp" />
	</LinearLayout>
	</LinearLayout>
	</LinearLayout>
	</androidx.cardview.widget.CardView>
	</LinearLayout>

11. nav_graph.xml

Tabel 11. Source Code Jawaban Soal 1 (nav_graph.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/homeFragment">
7	
8	<fragment
9	android:id="@+id/homeFragment"
10	android:name="com.example.modul4.HomeFragment"
11	android:label="Home">
12	<action
13	
14	android:id="@+id/action_homeFragment_to_detailFragment"
15	app:destination="@id/detailFragment" />
16	</fragment>
17	
18	<fragment
19	android:id="@+id/detailFragment"
20	android:name="com.example.modul4.DetailFragment"
21	android:label="Detail">
22	<argument
23	android:name="constellation"
24	app:argType="com.example.modul4.Constellation"

	<pre> /> </fragment> </navigation> </pre>
--	--

12. styles.xml

Tabel 12. Source Code Jawaban Soal 1 (styles.xml)

1	<code><?xml version="1.0" encoding="utf-8"?></code>
1	<code><resources></code>
2	<code> <style name="RoundedImageView" parent=""></code>
3	<code> <item name="cornerFamily">rounded</item></code>
4	<code> <item name="cornerSize">16dp</item></code>
5	<code> </style></code>
6	<code></resources></code>
7	

13. MainViewModel.kt

Tabel 13. Source Code Jawaban Soal 1 (MainViewModel.kt)

1	<code>package com.example.modul4</code>
2	
3	<code>import android.util.Log</code>
4	<code>import androidx.lifecycle.ViewModel</code>
5	<code>import kotlinx.coroutines.flow.MutableStateFlow</code>
6	<code>import kotlinx.coroutines.flow.StateFlow</code>
7	
8	<code>class MainViewModel : ViewModel() {</code>
9	
10	<code> private val _constellations =</code>
11	<code> MutableStateFlow<List<Constellation>>(emptyList())</code>
12	<code> val constellations: StateFlow<List<Constellation>> =</code>
13	<code> _constellations</code>
14	
15	<code> private val _selectedItem =</code>
16	<code> MutableStateFlow<Constellation?>(null)</code>
17	<code> val selectedItem: StateFlow<Constellation?> =</code>
18	<code> _selectedItem</code>
19	
20	<code> init {</code>
21	<code> val list = ConstellationData.listConstellations</code>
22	<code> _constellations.value = list</code>
23	<code> Log.d("ViewModel", "Data item masuk ke dalam list:</code>
24	<code> \$list")</code>
25	<code> }</code>
26	
27	<code> fun onDetailClicked(constellation: Constellation) {</code>
28	<code> _selectedItem.value = constellation</code>

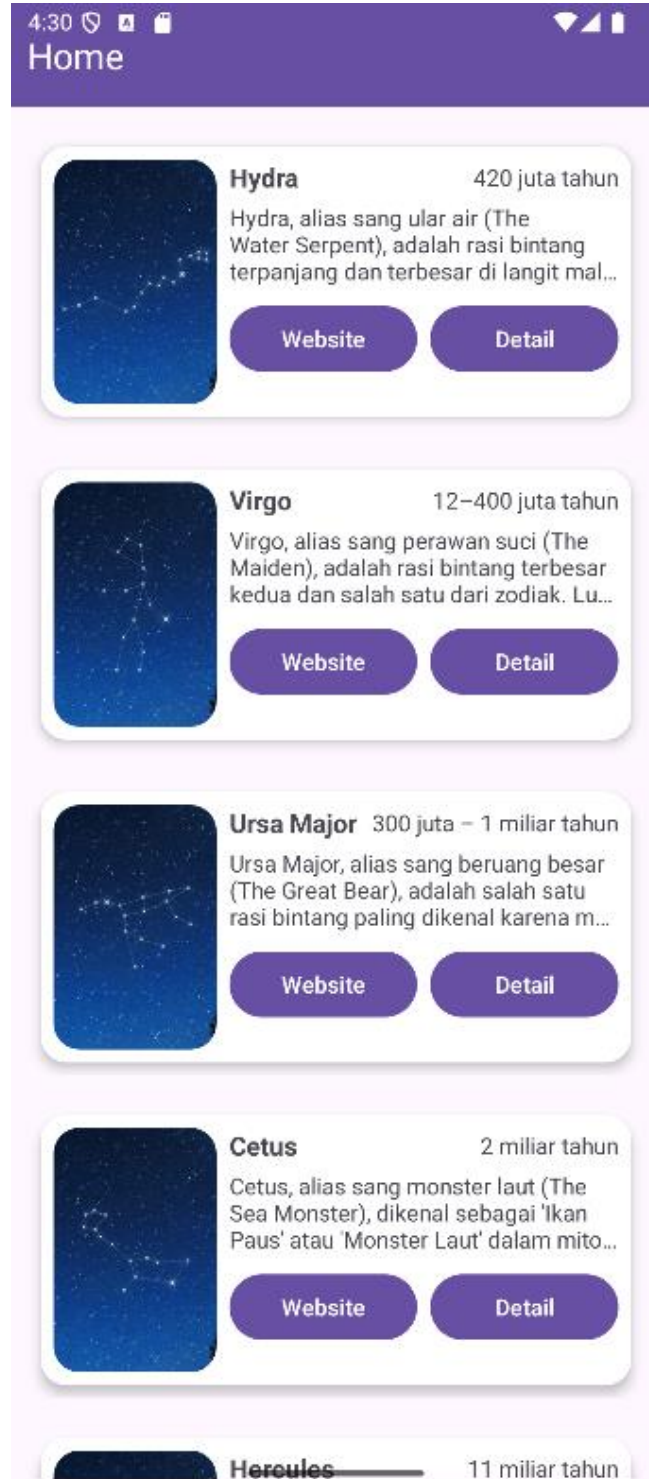
29	Log.d("ViewModel", "Tombol Detail ditekan. Item:
30	\$constellation")
	}
	fun onWebClicked(constellation: Constellation) {
	Log.d("ViewModel", "Tombol Explicit Intent ditekan.
	URL: \${constellation.webUrl}")
	}
	}

14. MainViewModelFactory.kt

Tabel 14. Source Code Jawaban Soal 1 (MainViewModelFactory.kt)

1	package com.example.modul4
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class MainViewModelFactory : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass:
8	Class<T>): T {
9	if
10	(modelClass.isAssignableFrom(MainViewModel::class.java)) {
11	return MainViewModel() as T
12	}
13	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

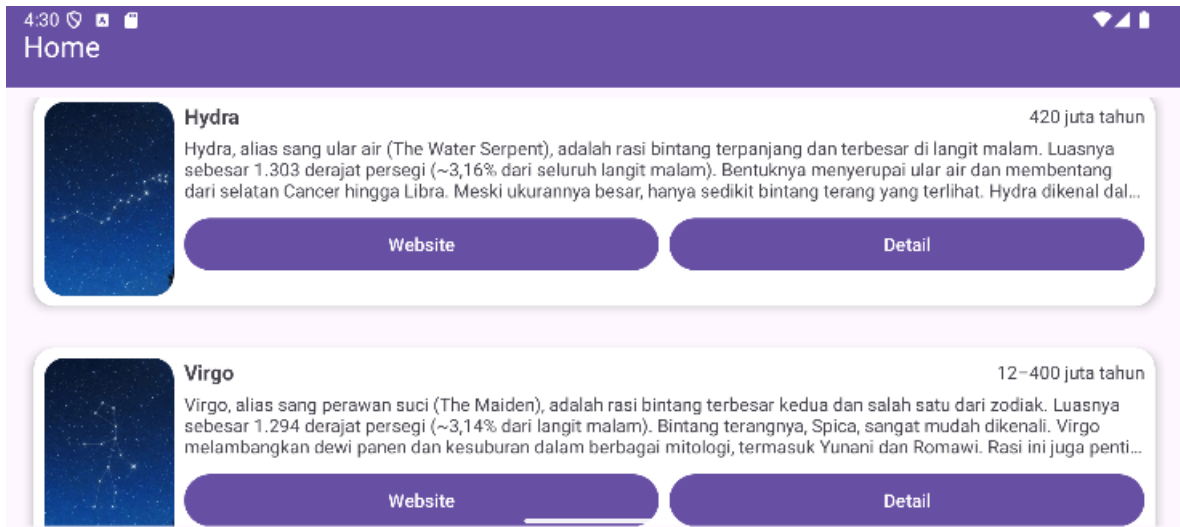
B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 (List Portrait)



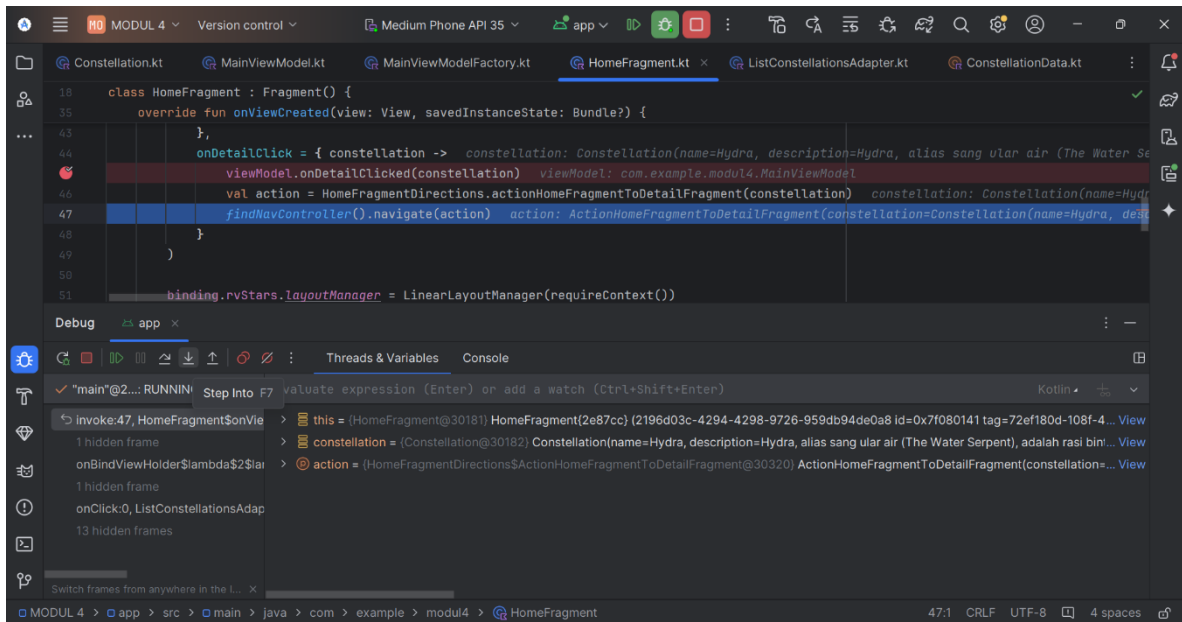
Gambar 2. Screenshot Hasil Jawaban Soal 1 (Detail Portrait)



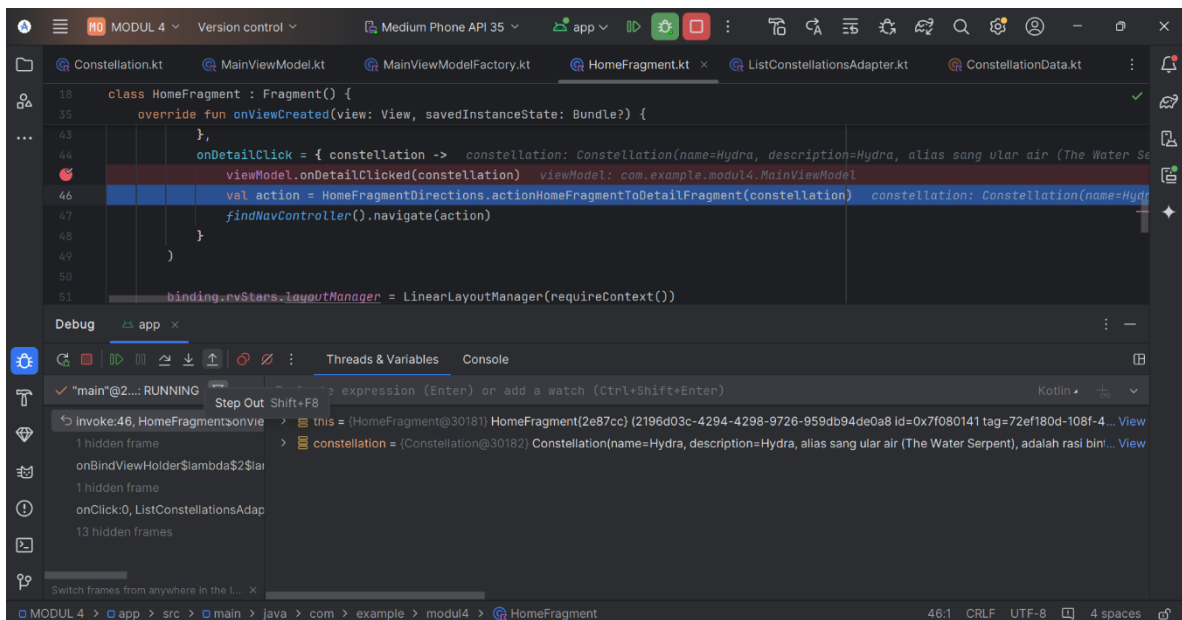
Gambar 3. Screenshot Hasil Jawaban Soal 1 (List Landscape)



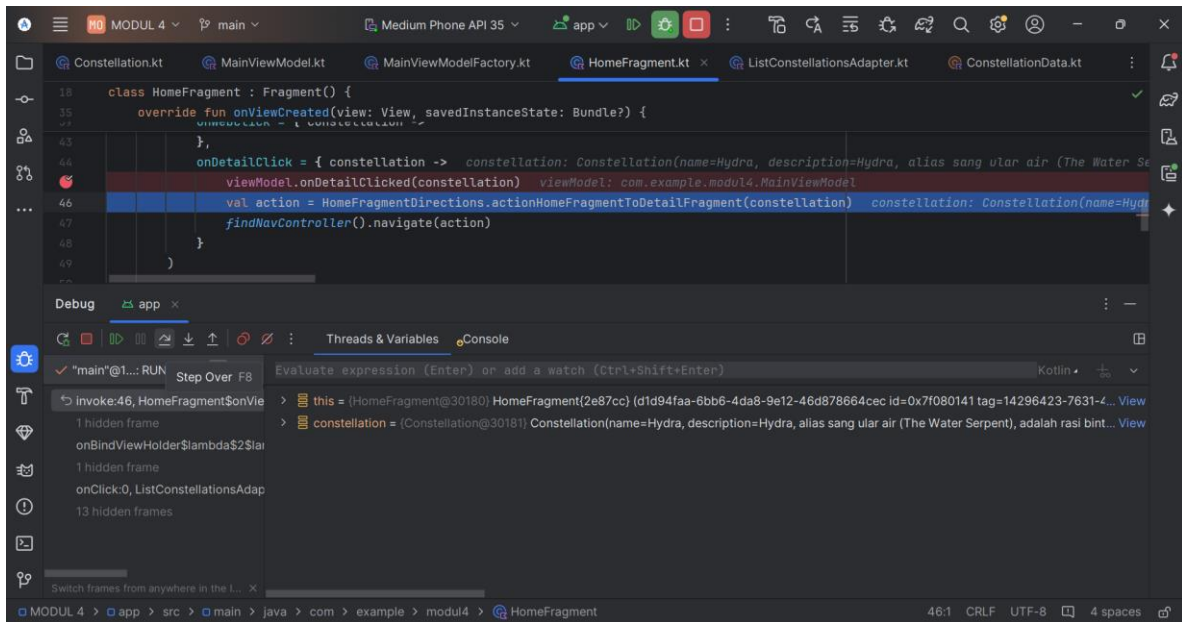
Gambar 4. Screenshot Hasil Jawaban Soal 1 (Detail Landscape)



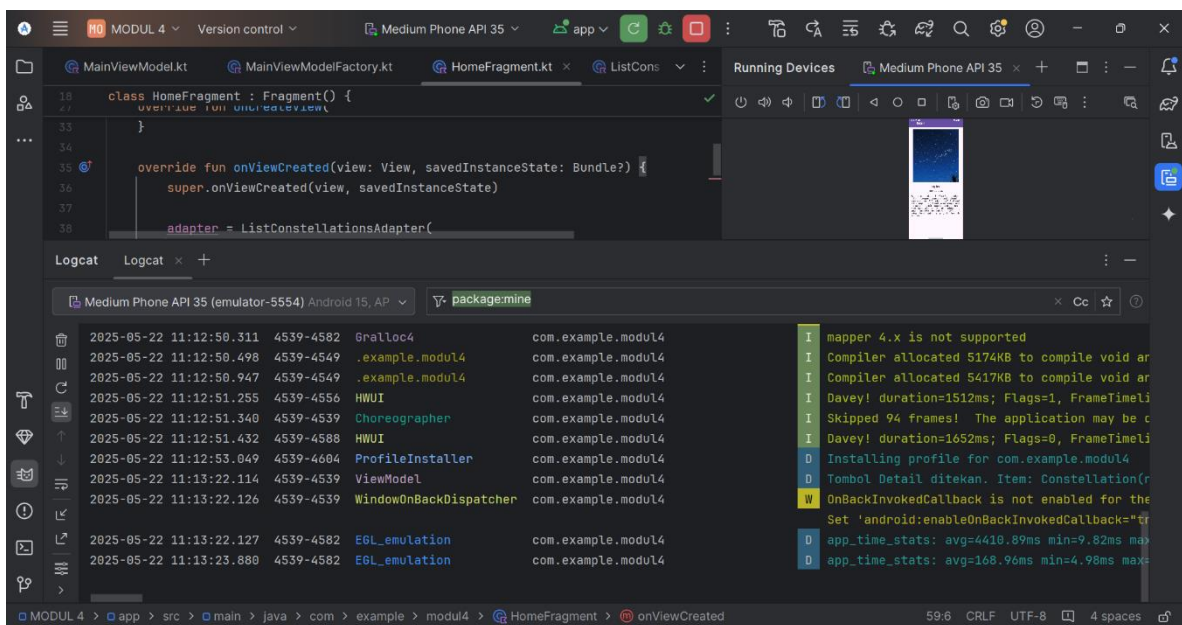
Gambar 5. Screenshot Hasil Jawaban Soal 1 (Step Into)



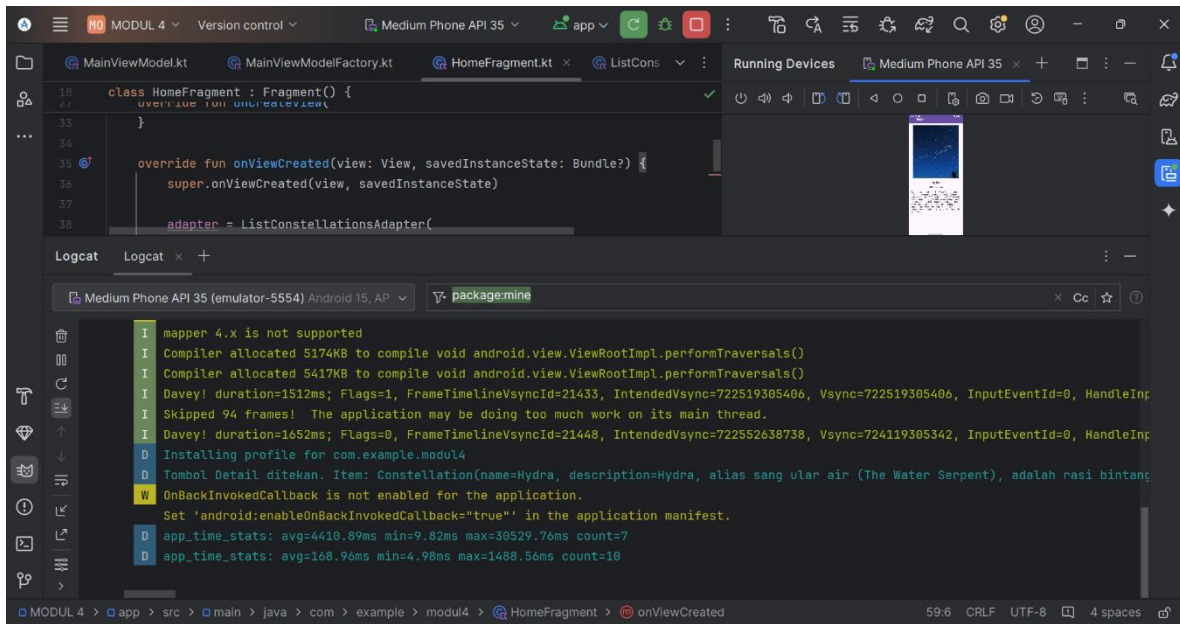
Gambar 6. Screenshot Hasil Jawaban Soal 1 (Step Out)



Gambar 7. Screenshot Hasil Jawaban Soal 1 (Step Over)



Gambar 8. Screenshot Hasil Jawaban Soal 1 (Logcat)



Gambar 9. Screenshot Hasil Jawaban Soal 1 (Validasi Logcat)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Bundle` untuk menyimpan status activity saat di-pause atau di-rotate.
- Pada line 4, mengimpor `AppCompatActivity` untuk menggunakan fitur kompatibilitas activity pada aplikasi.
- Pada line 5, mengimpor `NavHostFragment` untuk mendukung navigasi menggunakan fragment.
- Pada line 6, mengimpor `setupActionBarWithNavController` untuk mengatur aksi bar dengan navigasi controller.
- Pada line 7, mengimpor `ActivityMainBinding` untuk binding layout dari XML ke dalam activity.
- Pada line 9, mendeklarasikan kelas `MainActivity` yang mewarisi `AppCompatActivity`.

- Pada line 11, mendeklarasikan properti `binding` untuk akses komponen UI melalui `ViewBinding`.
- Pada line 13-17, membuat `onCreate`, meng-inflate layout dan mengatur konten view menggunakan binding.
- Pada line 19, menyetel `toolbar` sebagai action bar untuk activity.
- Pada line 21-23, mendapatkan `NavHostFragment` dan `NavController` untuk mendukung navigasi antar fragment.
- Pada line 25, menyiapkan action bar untuk bekerja dengan `NavController` yang ada.
- Pada line 28-32, terdapat Override `onSupportNavigateUp` untuk meng-handle navigasi kembali saat menekan tombol up di action bar.

2. Constellation.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Parcelable` untuk memungkinkan objek dikirim antar komponen Android.
- Pada line 4, mengimpor `Parcelize` untuk menyederhanakan implementasi `Parcelable` pada kelas.
- Pada line 6, terdapat anotasi `@Parcelize` digunakan untuk otomatis mengimplementasikan `Parcelable`.
- Pada line 7-12, mendefinisikan kelas data `Constellation` dengan properti: `name`, `description`, `year`, `imageResId`, dan `webUrl`.
- Pada line 13, terdapat `Parcelable` yang artinya kelas `Constellation` mengimplementasikan `Parcelable` untuk mendukung pengiriman objek antar komponen.

3. ConstellationData.kt:

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.

- Pada line 3, mendeklarasikan objek `ConstellationData`, yang menyimpan data statis tentang konstelasi bintang.
- Pada line 4, mendeklarasikan properti `listConstellations` sebagai daftar (list) objek `Constellation`.
- Pada line 5-39, menambahkan beberapa objek `Constellation` ke dalam `listConstellations`, masing-masing dengan `name` (nama konstelasi), `description` (deskripsi singkat mengenai konstelasi), `imageResId` (ID gambar konstelasi), `year` (perkiraan usia konstelasi), dan `webUrl` (URL untuk informasi lebih lanjut mengenai konstelasi).

4. **HomeFragment.kt:**

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Intent` dari Android, yang digunakan untuk melakukan operasi seperti membuka browser atau berpindah Activity.
- Pada line 4, mengimpor `Uri` dari Android, yang digunakan untuk memarsing dan merepresentasikan URL.
- Pada line 5, mengimpor `Bundle` yang digunakan untuk menyimpan dan memulihkan data saat fragment dibuat.
- Pada line 6, mengimpor `LayoutInflater` yang digunakan untuk menampilkan layout XML ke dalam objek `View`.
- Pada line 7, mengimpor `View`, komponen dasar antarmuka pengguna Android.
- Pada line 8, mengimpor `ViewGroup`, elemen yang bisa berisi banyak `View`.
- Pada line 9, mengimpor `Fragment`, kelas dasar untuk membuat bagian UI yang modular di Android.
- Pada line 10, mengimpor fungsi `viewModels`, digunakan untuk memperoleh instance `ViewModel` dengan pendekatan delegasi.
- Pada line 11, mengimpor `lifecycleScope` untuk menjalankan coroutine yang mengikuti lifecycle `Fragment`.

- Pada line 12, mengimpor `findNavController` untuk melakukan navigasi antar-fragment.
- Pada line 13, mengimpor `LinearLayoutManager` yang digunakan untuk menampilkan item di `RecyclerView` secara vertikal.
- Pada line 14, mengimpor binding otomatis `FragmentHomeBinding`, yang dibuat dari layout XML `fragment_home.xml`.
- Pada line 15, mengimpor fungsi `collectLatest` dari Kotlin Flow untuk mengamati dan mengambil data terbaru.
- Pada line 16, mengimpor `launch` dari `Coroutine` untuk menjalankan coroutine di dalam `lifecycleScope`.
- Pada line 18, mendeklarasikan kelas `HomeFragment` yang merupakan turunan dari `Fragment`.
- Pada line 20, mendeklarasikan variabel `_binding` sebagai nullable `FragmentHomeBinding`, yang digunakan untuk mengakses elemen UI di XML.
- Pada line 21, mendefinisikan properti `binding` sebagai non-nullable dengan menggunakan `!!`, sehingga kita bisa langsung mengakses binding dengan aman selama view masih aktif.
- Pada line 23, membuat instance `viewModel` dari `MainViewModel` menggunakan delegasi by `viewModels` dan `MainViewModelFactory`.
- Pada line 25, mendeklarasikan variabel `adapter` dari tipe `ListConstellationsAdapter` untuk menampilkan daftar konstelasi.
- Pada line 27-33, mendefinisikan fungsi `onCreateView`, yang dipanggil saat fragment membuat UI-nya. `inflater` digunakan untuk meng-inflate layout XML ke dalam objek `View`.
- Pada line 31, `FragmentHomeBinding.inflate()` digunakan untuk menghubungkan XML layout dengan objek Kotlin.
- Pada line 32, mengembalikan `binding.root` sebagai tampilan utama dari fragment.

- Pada line 35-59, mendefinisikan fungsi `onViewCreated`, yang dipanggil setelah fragment view selesai dibuat.
- Pada line 38-49, menginisialisasi adapter dari `ListConstellationsAdapter` dengan dua lambda:
 - `onWebClick`: Menangani klik tombol web. Memanggil fungsi `onWebClicked()` di `ViewModel`, lalu membuat Intent eksplisit untuk membuka browser ke URL konstelasi.
 - `onDetailClick`: Menangani klik tombol detail. Memanggil fungsi `onDetailClicked()` di `ViewModel`, lalu membuat navigasi ke `DetailFragment` menggunakan `Safe Args`.
- Pada line 51-52, mengatur `layoutManager` dari `RecyclerView` agar menampilkan daftar item secara vertikal.
- Pada line 54-58, menggunakan `coroutine(lifecycleScope.launch)` untuk mengamati data `constellations` dari `ViewModel`. Jika ada perubahan data, maka akan diserahkan ke `adapter.submitList()` untuk ditampilkan ke UI.
- Pada line 61-64, mendefinisikan `onDestroyView`, yang dipanggil saat fragment dihancurkan. Di sini binding di-set ke null untuk mencegah memory leak.

5. DetailFragment.kt

- Pada line 1, mendeklarasikan nama *package* untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, diimpor `android.os.Bundle` untuk menangani data yang dikirim antar komponen dalam aplikasi, seperti data yang dikirim saat navigasi antar fragment.
- Pada line 4, diimpor `android.view.LayoutInflater` untuk meng-inflate layout XML ke dalam objek View di dalam fragment.
- Pada line 5, diimpor `android.view.View` untuk merepresentasikan elemen tampilan di dalam fragment.

- Pada line 6, diimpor `android.view.ViewGroup` untuk merepresentasikan container dari elemen tampilan dalam fragment.
- Pada line 7, diimpor `androidx.fragment.app.Fragment` untuk memungkinkan class ini mewarisi fungsionalitas dari fragment, seperti menangani lifecycle dan tampilan.
- Pada line 8, diimpor `androidx.navigation.fragment.navArgs` untuk mengakses argumen yang dikirim saat navigasi antar fragment menggunakan komponen Navigation.
- Pada line 9, diimpor `com.bumptech.glide.Glide` untuk memudahkan memuat dan menampilkan gambar dari URL atau sumber lainnya ke dalam `ImageView`.
- Pada line 10, diimpor `com.example.modul4.databinding.FragmentDetailBinding` untuk mengakses elemen-elemen di layout `fragment_detail.xml` menggunakan `ViewBinding`.
- Pada line 12, mendeklarasikan kelas `DetailFragment`, yang menampilkan detail konstelasi saat item dipilih dari `HomeFragment`.
- Pada line 14, mendeklarasikan properti `_binding` untuk mengakses komponen UI menggunakan `ViewBinding`.
- Pada line 15, mendeklarasikan properti `binding` untuk memudahkan akses ke `FragmentDetailBinding`.
- Pada line 10, menggunakan `navArgs` untuk mengambil data `constellation` yang dipassing dari `HomeFragment`.
- Pada line 18-24, membuat `onCreateView`, meng-inflate layout dan mengembalikan root view untuk fragment.
- Pada line 26-38, membuat `onViewCreated`, mengatur tampilan detail dengan data yang diterima, seperti menampilkan nama, deskripsi, dan tahun konstelasi, lalu, menggunakan `Glide` untuk memuat gambar konstelasi ke dalam `detailImage`.

- Pada line 40-43, membuat `onDestroyView`, membersihkan binding untuk menghindari memory leak.

6. **ListConstellationsAdapter.kt:**

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `LayoutInflater`, digunakan untuk meng-inflate layout XML menjadi objek `View`.
- Pada line 4, mengimpor `ViewGroup`, elemen yang bisa berisi banyak `View` dan biasanya menjadi parent layout.
- Pada line 5, mengimpor `DiffUtil`, utility untuk membandingkan data lama dan baru dalam `ListAdapter`.
- Pada line 6, mengimpor `ListAdapter`, adapter khusus yang otomatis mengelola perbedaan data menggunakan `DiffUtil`.
- Pada line 7, mengimpor `RecyclerView`, komponen untuk menampilkan daftar data secara efisien.
- Pada line 8, mengimpor `Glide`, library eksternal untuk memuat dan menampilkan gambar dari URL atau resource.
- Pada line 9, mengimpor `StarItemBinding`, kelas binding otomatis dari layout XML `star_item.xml`.
- Pada line 11-14, mendefinisikan kelas `ListConstellationsAdapter` yang merupakan turunan dari `ListAdapter`.
 - Adapter ini menerima dua parameter lambda: `onWebClick` dan `onDetailClick` untuk menangani event klik tombol Web dan Detail.
 - Constellation adalah tipe data yang ditampilkan dalam daftar.
 - `DIFF_CALLBACK` digunakan untuk mendeteksi perubahan data dengan efisien.

- Pada line 16, mendefinisikan inner class `ViewHolder` yang menerima `StarItemBinding`, digunakan untuk merepresentasikan setiap item di `RecyclerView`.
- Pada line 18–21, mengoverride fungsi `onCreateViewHolder`, yang bertanggung jawab membuat tampilan (View) untuk setiap item list.
 - `LayoutInflater.from(parent.context)` digunakan untuk meng-inflate layout XML.
 - `StarItemBinding.inflate()` menghasilkan objek binding dari layout `star_item.xml`.
- Pada line 23-34, mengoverride fungsi `onBindViewHolder`, untuk menghubungkan data ke tampilan UI berdasarkan posisi item.
 - `getItem(position)` mengambil data konstelasi berdasarkan posisi.
 - Dengan `with(holder.binding)`, kita mengakses elemen UI seperti `tvName`, `tvDescription`, dll.
 - `Glide.with(img.context).load(item.imageResId).into(img)` digunakan untuk memuat gambar ke dalam `ImageView`.
 - `btnWeb.setOnClickListener` dan `btnDetail.setOnClickListener` menetapkan aksi klik untuk masing-masing tombol.
- Pada line 36-46, mendeklarasikan objek `DIFF_CALLBACK` sebagai implementasi `DiffUtil.ItemCallback<Constellation>`.
 - Fungsi `areItemsTheSame` membandingkan apakah dua item memiliki ID atau key yang sama (di sini digunakan `name`).
 - Fungsi `areContentsTheSame` membandingkan isi dari dua item, apakah benar-benar identik (menggunakan `==`).

7. MainViewModel.kt:

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `Log` dari Android, digunakan untuk mencetak pesan ke Logcat, biasanya untuk keperluan debugging.
- Pada line 4, mengimpor `ViewModel` dari Jetpack Lifecycle, yang digunakan untuk menyimpan dan mengelola data UI agar tetap bertahan ketika terjadi perubahan konfigurasi (misalnya rotasi layar).
- Pada line 5, mengimpor `MutableStateFlow` dari Kotlin Coroutines, yang merupakan state holder yang bisa diubah dan diamati untuk reactive programming.
- Pada line 6, mengimpor `StateFlow`, yaitu versi read-only dari `MutableStateFlow` yang bisa diamati oleh UI.
- Pada line 8, mendeklarasikan kelas `MainViewModel` yang merupakan subclass dari `ViewModel`.
- Pada line 10, mendeklarasikan properti privat `_constellations` bertipe `MutableStateFlow<List<Constellation>>` dan diinisialisasi dengan list kosong. Ini digunakan untuk menyimpan daftar konstelasi secara reaktif.
- Pada line 11, mendeklarasikan properti publik `constellations` sebagai `StateFlow<List<Constellation>>`, yaitu versi hanya-baca dari `_constellations` agar data tidak bisa diubah dari luar kelas.
- Pada line 13, mendeklarasikan properti privat `_selectedItem` bertipe `MutableStateFlow<Constellation?>` dan diinisialisasi dengan `null`. Ini digunakan untuk menyimpan konstelasi yang sedang dipilih oleh user.
- Pada line 14, mendeklarasikan properti publik `selectedItem` sebagai `StateFlow<Constellation?>`, yaitu versi read-only dari `_selectedItem`.
- Pada line 16, mendeklarasikan blok `init`, yaitu blok inisialisasi yang akan dijalankan saat `MainViewModel` pertama kali dibuat.

- Pada line 17, mengambil data konstelasi dari `ConstellationData.listConstellations` dan menyimpannya dalam variabel `list`.
- Pada line 18, mengatur nilai `_constellations` dengan `list` agar data konstelasi bisa diamati dari UI.
- Pada line 19, mencetak log dengan tag "ViewModel" dan pesan bahwa data item sudah dimasukkan ke dalam `list`.
- Pada line 22, mendefinisikan fungsi `onDetailClicked` dengan parameter `constellation` yang akan dipanggil ketika tombol detail diklik oleh pengguna.
- Pada line 23, mengubah nilai `_selectedItem` menjadi `constellation` yang dipilih, sehingga UI bisa merespons perubahan tersebut.
- Pada line 24, mencetak log bahwa tombol detail telah ditekan beserta item yang dipilih.
- Pada line 27, mendefinisikan fungsi `onWebClicked` dengan parameter `constellation` yang akan dipanggil ketika tombol menuju halaman web diklik.
- Pada line 28, mencetak log bahwa tombol web ditekan dan menampilkan URL dari konstelasi tersebut.

8. **MainViewModelFactory.kt:**

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul4`.
- Pada line 3, mengimpor `ViewModel` dari Jetpack Lifecycle, yang merupakan superclass untuk semua `ViewModel`.
- Pada line 4, mengimpor `ViewModelProvider` dari Jetpack Lifecycle, yang digunakan untuk membuat dan mengelola objek `ViewModel` dengan cara yang aman terhadap siklus hidup (lifecycle-aware).
- Pada line 6, mendeklarasikan kelas `MainViewModelFactory` yang mengimplementasikan `ViewModelProvider.Factory`. Kelas ini bertugas membuat instance `MainViewModel`.

9. `activity_main.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5-6, terdapat `layout_width` & `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8-14, membuat `AppBarLayout` yang ditambahkan sebagai action bar custom:
 - `layout_height` mengikuti tinggi standar action bar (`?attr/actionBarSize`).
 - `background` mengikuti warna utama tema (`colorPrimary`).
 - `elevation 4dp` memberi efek bayangan.
 - `titleTextColor` di-set ke putih.
- Pada line 16-23, terdapat `FragmentContainerView` untuk menampilkan fragment berdasarkan `Navigation Component`:
 - `layout_marginTop` disesuaikan dengan tinggi action bar supaya fragment tidak tertutup toolbar.
 - `android:name` menunjuk ke `NavHostFragment`, elemen utama navigasi.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama.
 - `app:navGraph` menunjuk ke file navigasi `nav_graph.xml`.
- Pada line 25, penutup `CoordinatorLayout`.

10. `fragment_home.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat root `FrameLayout` digunakan sebagai wadah layout fragment, dengan ID `homeFragment`, ukuran penuh, dan padding 8dp.

- Pada line 8-12, terdapat `RecyclerView` dengan ID `rvStars` digunakan untuk menampilkan daftar rasi bintang dalam list scrollable, memenuhi seluruh ruang yang tersedia dan `clipToPadding` diset false agar konten tetap bisa tampil meski melebihi padding.

11. `fragment_detail.xml`:

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat deklarasi `ScrollView` sebagai root layout yang memungkinkan isi halaman dapat discroll secara vertikal. Ini penting karena kontennya bisa lebih panjang dari tinggi layar.
- Pada line 3, terdapat atribut `xmlns` untuk mendefinisikan namespace Android dan `app`, yang wajib ada agar atribut khusus (seperti `app:shapeAppearanceOverlay`) bisa digunakan.
- Pada line 4, terdapat `android:id="@+id/detailFragment"` sebagai ID unik untuk `ScrollView`, meskipun tidak wajib jika tidak digunakan di kode.
- Pada line 6-7, `layout_width` dan `layout_height` diatur `match_parent` agar `ScrollView` mengisi seluruh layar.
- Pada line 7, `android:padding="16dp"` memberi jarak di sekeliling isi layout supaya tidak terlalu mepet ke tepi layar.
- Pada line 9-13, terdapat `LinearLayout` dengan orientasi vertikal dan `gravity center_horizontal`, artinya elemen di dalamnya akan ditumpuk ke bawah dan disejajarkan di tengah horizontal.
- Pada line 15-21, terdapat `ImageView` (`@id/detailImage`) untuk menampilkan gambar rasi bintang. Lebaranya 350dp dan tingginya 400dp, `scaleType="centerCrop"` agar gambar tetap proporsional meski di-crop, dan `layout_marginBottom="16dp"` memberi jarak ke bawah. Atribut `app:shapeAppearanceOverlay` digunakan untuk membuat gambar tampil dengan sudut membulat (rounded).

- Pada line 23–30, terdapat `TextView (@id/detailName)` untuk menampilkan nama rasi bintang, teks ditampilkan bold, ukuran besar (22sp), dan diberi jarak bawah 8dp agar rapi.
- Pada line 32-38, terdapat `TextView (@id/detailYear)` untuk menampilkan tahun atau data waktu terkait rasi bintang. Ukuran teks 16sp dan jarak bawah 16dp.
- Pada line 40-45, terdapat `TextView (@id/detailDescription)` yang menampilkan deskripsi panjang tentang rasi bintang. `layout_width` diatur `match_parent` supaya teks memenuhi lebar layar, dan ukuran teks tetap 16sp agar mudah dibaca.

12. star_item.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, deklarasi root `LinearLayout` vertikal yang membungkus satu item rasi bintang. Layout-nya `match_parent` lebar, tinggi menyesuaikan isi. `Padding` 8dp biar ga terlalu mepet tepi.
- Pada line 5, tambahan namespace `app` agar bisa memakai properti khusus seperti `shapeAppearanceOverlay`.
- Pada line 9-14, `CardView` untuk memberi tampilan seperti kartu. Diberi `margin` 8dp, sudut melengkung (`cardCornerRadius=16dp`), dan bayangan (`cardElevation=4dp`) biar ada efek ngambang.
- Pada line 16-20, `LinearLayout` horizontal di dalam `CardView` untuk menampilkan dua sisi, yaitu kiri (gambar) & kanan (info). `Padding` 8dp untuk ruang dalam.
- Pada line 22-28, `ShapeableImageView (@id/img)` untuk menampilkan gambar rasi. Ukuran 100x150dp, crop tengah, dan dibuat rounded lewat `shapeAppearanceOverlay`.
- Pada line 30-34, `LinearLayout` vertikal dengan `layout_weight="1"` supaya mengisi sisa ruang di sebelah kanan gambar.

- Pada line 36-40, `LinearLayout` horizontal untuk baris pertama info: nama rasi dan tahun. `gravity="center_vertical"` agar teks sejajar tengah secara vertikal.
- Pada line 42-49, `TextView (@id/tvName)` untuk nama rasi bintang, pakai `layout_weight="1"` agar fleksibel. Ukuran teks 16sp, bold, responsif ke sisa ruang.
- Pada line 51-56, `TextView (@id/tvYear)` untuk menampilkan tahun, posisinya di sebelah kanan nama.
- Pada line 59-67, `TextView (@id/tvDescription)` untuk deskripsi singkat. Margin atas 4dp, teks 14sp, maksimal 3 baris, dan `ellipsize="end"` artinya kalau terlalu panjang, akan dipotong dan dikasih tanda titik tiga (...).
- Pada line 69-89, `LinearLayout` horizontal untuk dua tombol: Website & Detail. Margin atas 8dp biar terpisah dari deskripsi.
- Pada line 75-80, `Button (@id/btnWeb)` untuk buka URL website rasi. Pakai `layout_weight="1"` untuk membagi ruang setengah layar.
- Pada line 82-88, `Button (@id/btnDetail)` untuk masuk ke halaman detail rasi. Dikasih `layout_marginStart="8dp"` agar tidak mepet ke tombol sebelumnya.

13. nav_graph.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2, terdapat elemen root `<navigation>` yang menjafi tempat mengatur alur navigasi antar fragment. Terdapat namespace android untuk atribut standar Android.
- Pada line 3, namespace app untuk properti khusus dari Android Jetpack Navigation.
- Pada line 4, `android:id="@+id/nav_graph"` adalah ID untuk grafik navigasi ini.
- Pada line 5, `app:startDestination="@id/homeFragment"` menunjukkan bahwa homeFragment adalah layar awal saat aplikasi dibuka.

- Pada line 7-14, deklarasi fragment untuk homeFragment (@+id/homeFragment).
- Pada line 9, menunjukkan bahwa kelas fragment-nya adalah `com.example.starconstellations.HomeFragment`
- Pada line 10, menunjukkan bahwa label yang muncul di toolbar adalah "Home"
- Pada line 11-13, terdapat `<action>` yang mengatur aksi `action_homeFragment_to_detailFragment` untuk navigasi dari Home ke Detail. `app:destination="@id/detailFragment"` menunjukkan tujuan navigasinya
- Pada line 14-18, deklarasi fragment untuk detailFragment (@+id/detailFragment).
- Pada line 18, menunjukkan bahwa kelas fragment-nya: `com.example.starconstellations.DetailFragment`
- Pada line 19, menunjukkan bahwa nama label adalah "Detail"
- Pada line 20-22, terdapat argument bernama "constellation". Hal ini digunakan untuk mengirim data objek bertipe `com.example.starconstellations.Constellation` dari Home ke Detail.

14. styles.xml:

- Pada line 1, deklarasi encoding XML UTF-8.
- Pada line 2, terdapat `<resources>` yang merupakan root element untuk menyimpan semua resource seperti style, color, dll.
- Pada line 3, deklarasi style baru bernama `RoundedImageView`, tidak mewarisi style lain (`parent=""`).
- Pada line 4, terdapat `cornerFamily` yang di-set ke `rounded` artinya semua sudut akan dibulatkan.
- Pada line 5, `cornerSize` di-set ke `16dp` artinya radius lengkung sudut sebesar `16dp`.

- Pada line 7, mengoverride fungsi `create` dari interface `ViewModelProvider.Factory`, yang akan dipanggil saat sistem membutuhkan instance `ViewModel`.
- Pada line 8, memeriksa apakah `modelClass` yang diminta adalah `MainViewModel` atau subclass-nya menggunakan `isAssignableFrom`.
- Pada line 9, jika benar, maka membuat instance `MainViewModel` dan mengembalikannya sebagai objek bertipe `T`.
- Pada line 11, jika `modelClass` bukan `MainViewModel`, maka melempar exception `IllegalArgumentException` dengan pesan “Unknown ViewModel class”.

15. Penjelasan Fungsi Debugger

- Debugger digunakan untuk memeriksa alur program dan nilai variabel secara detail.
- Penggunaan debugger memungkinkan saya untuk melihat alur eksekusi kode, memeriksa isi variabel, dan mengetahui letak kesalahan secara spesifik.
- Breakpoint adalah titik henti sementara dalam kode program. Saat eksekusi mencapai titik ini, program akan di-pause, memungkinkan kita untuk memeriksa isi dan alur.
- Fitur debugger sangat penting saat pengembangan aplikasi agar bisa memahami dan mengevaluasi bagaimana logika aplikasi bekerja secara real-time.
- ☐ Step Into mengeksplorasi isi fungsi, Step Over menjalankan tanpa masuk fungsi, dan Step Out keluar dari fungsi ke level di atasnya

16. Cara Menggunakan Debugger

- Saya menambahkan breakpoint pada baris `viewModel.onDetailClicked(constellation)` di file `HomeFragment.kt`, karena baris ini menangani event saat user klik tombol "Detail".

- Saya menjalankan aplikasi dengan meng-klik icon debug. Lalu, saya meng-klik tombol "Detail" di list item dan aplikasi berhenti di breakpoint yang sudah saya pasang.
- Setelah aplikasi terbuka dan saya klik tombol "Detail", debugger berhenti di breakpoint, menunjukkan bahwa event telah berhasil dipicu.
- Saat program berhenti di breakpoint, saya:
 - Menggunakan Step Into (F7) untuk masuk ke dalam fungsi `onDetailClicked()` dan melihat bagaimana ViewModel memproses data.
 - Menggunakan Step Over (F8) untuk menjalankan baris kode saat ini tanpa masuk ke dalam fungsi.
 - Menggunakan Step Out (Shift + F8) untuk keluar dari fungsi dan kembali ke pemanggil sebelumnya (Fragment).
- Saya juga menggunakan panel Variables di Debugger untuk melihat isi objek constellation seperti `name`, `description`, dan `webUrl`.

17. Penjelasan Fitur Step Into, Step Over, dan Step Out

- Step Into (F7): Digunakan untuk masuk ke dalam metode atau fungsi yang sedang dipanggil. Dalam kasus ini, digunakan untuk masuk ke fungsi `onDetailClicked()` di `MainViewModel`.
- Step Over (F8): Menjalankan baris kode saat ini dan langsung melanjutkan ke baris berikutnya tanpa masuk ke dalam fungsi yang dipanggil.
- Step Out (Shift + F8): Keluar dari fungsi saat ini dan kembali ke pemanggilnya. Berguna saat kita sudah selesai memeriksa bagian dalam dari fungsi dan ingin melanjutkan ke level di atasnya.
- Saya juga melihat nilai dari variabel constellation melalui panel debugger, yang menunjukkan data item yang sedang diproses.

D. Tautan Git

[desyyn/PEMROGRAMAN-MOBILE](https://github.com/desyyn/PEMROGRAMAN-MOBILE)

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Application adalah kelas dasar dari Android yang mewakili seluruh aplikasi, berfungsi sebagai titik masuk utama dan tempat untuk menginisialisasi dan mengelola status aplikasi secara global. Kelas ini dijalankan satu kali saat aplikasi pertama kali diluncurkan dan tetap aktif selama proses aplikasi berjalan. Secara teknis, Application adalah subclass dari `android.app.Application`. Kelas ini dibuat sebelum komponen lain seperti Activity dan Service, sehingga menyediakan tempat yang tepat untuk menginisialisasi hal-hal yang dibutuhkan oleh seluruh aplikasi, seperti menginisialisasi database atau membuat objek global. Beberapa fungsi utama dari application class adalah sebagai tempat untuk menginisialisasi hal-hal global seperti ViewModelFactory, Library, atau Dependency Injection (Hilt, Dagger), menyimpan state atau variabel global yang ingin diakses dari berbagai komponen aplikasi, membuat sub class dari Application Class dan menentukan nama sub class ini dalam `AndroidManifest.xml` sebagai `android:name` atribut dalam tag `<application>`, serta dapat menyediakan `onCreate()` yang dipanggil pertama kali saat aplikasi dibuka, bahkan sebelum MainActivity. Application class ini cocok untuk setup logger global, crash reporting (misalnya Firebase Crashlytics), atau exception handler.